VISHNU INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FLUTTER MINI PROJECT LABORATORY REPORT

StockBasket – Grocery Inventory System

"Manage Smart. Stock Better."

Submitted By: Mutakani Dinni Sravya Roll No: 24PA1A05F7 B.Tech – CSE	**Subject / Laboratory:** Flutter & Mobile Application Development Laboratory Academic Year: 2025–2026
GitHub Source Repository: https://github.com/MUTAKANI-DINNI-SRAVYA/flutterproject.git	**Framework & Language:** Flutter (Material 3) & Dart Architecture: Clean Layered State Pattern

Bhimavaram, Andhra Pradesh, India

2. Student Details and Project Profile

This laboratory mini-project report documents the complete architectural design, data modeling, reactive state management, user interface implementation, and local data persistence of StockBasket – a production-ready mobile and web application built using the Flutter UI framework and Dart programming language.

Student Name	Mutakani Dinni Sravya
Roll Number	24PA1A05F7
Academic Program	Bachelor of Technology (B.Tech)
Department	Computer Science and Engineering (CSE)
Institution	Vishnu Institute of Technology (Autonomous), Bhimavaram
Laboratory Course	Flutter & Mobile Application Development Laboratory
Academic Year	2025–2026
Project Title	StockBasket – Grocery Inventory System
Project Tagline	Manage Smart. Stock Better.
Framework & Tooling	Flutter 3.x with Material 3 Design System
Programming Language	Dart (Strongly-typed, sound null safety)
State Management	ChangeNotifier & InheritedNotifier (InventoryScope)
Data Persistence	shared_preferences with JSON Serialization & In-Memory Fallback
GitHub Repository	https://github.com/MUTAKANI-DINNI-SRAVYA/flutterproject.git

Project Scope and Purpose: StockBasket is engineered to solve acute inventory challenges faced by retail grocery shop owners, local kirana merchants, supermarket department managers, and academic demonstrators. The system empowers users to digitally organize grocery stock, compute financial retail valuations and projected profits, receive proactive replenishment alerts when items drop below safety thresholds, track expiry dates to prevent food waste, and perform one-tap in-line stock increments and decrements directly from their mobile or web devices.

3. Project Description

3.1 Introduction

Grocery retail is a hyper-dynamic commerce environment characterized by high transaction frequency, short product shelf-lives, and slim profit margins. Unlike durable merchandise such as apparel or consumer electronics, grocery inventory encompasses perishable items (produce, dairy, fresh bakery items) that depreciate quickly if not sold before designated expiration dates. As retail operations expand, paper registers and spreadsheet-based ledger entries rapidly break down, leading to inaccurate count reconciliation, unrecognized theft or spoilage, and stockouts of critical consumer staples.

StockBasket bridges this technological divide by introducing an elegant, reactive, and portable digital inventory management system. Built with Flutter, it operates smoothly across mobile phones, tablets, and desktop browsers without requiring complicated external database configuration or continuous cloud internet connectivity.

3.2 Problem Statement

Manual Deficiencies: Traditional grocery store inventory management relies heavily on manual counting, ledger book entries, or fragmented electronic spreadsheets. These outdated mechanisms introduce severe operational liabilities:

- **Untracked Out-of-Stock Incidents:** Store managers frequently discover stockouts only after customers inquire about missing products, leading to customer churn and direct revenue loss.
- **Perishable Spoilage and Food Waste:** Failure to systematically record and monitor expiration dates results in spoiled goods that must be written off as financial losses, conflicting directly with sustainable consumption guidelines.
- **Laborious Valuation Calculations:** Computing total store investment cost, retail inventory valuation, and prospective gross margins requires manual arithmetic across hundreds of SKU items, making real-time financial audits impossible.
- **Error-Prone Stock Updates:** Requiring a store clerk to open a complex edit form just to reflect the sale or arrival of five cans of produce introduces latency and human error.

3.3 Objectives

The StockBasket project was designed and implemented to accomplish the following core technical and operational objectives:

- **Centralized Product Directory:** Establish a digital, search-optimized repository of grocery products indexed by Name, SKU/Barcode, Category, and Supplier.
- **Proactive Reorder Monitoring:** Implement dynamic low-stock and out-of-stock warning algorithms based on user-defined minimum threshold limits.
- **Shelf-Life and Expiry Tracking:** Incorporate expiration date tracking that proactively highlights products expiring within 14 days or already expired.

- **Real-Time Financial Analytics:** Calculate live retail valuation, total investment cost, projected gross profit, and store margin percentages automatically.
- **One-Tap In-Line Adjustments:** Enable rapid stock increments and decrements directly from summary list cards and a specialized Quick Stock dialog.
- **Resilient Local Data Persistence:** Utilize SharedPreferences with robust JSON serialization and an automated in-memory fallback layer to guarantee crash-free execution in all sandbox environments.

3.4 Target Users

User Profiles: The application is tailored for three distinct user groups:

- **Independent Kirana & Retail Shop Owners:** Small business proprietors who require a fast, clean, zero-maintenance inventory assistant that operates directly on personal smartphones without enterprise subscription fees.
- **Supermarket Department Managers:** Supermarket supervisors managing individual sections (such as Dairy, Bakery, or Fresh Produce) who need instant stock-health metrics and bulk replenishment controls.
- **Academic Evaluators & Mobile Developers:** College faculty, lab examiners, and engineering students evaluating clean Flutter architecture, InheritedNotifier state design, and Material 3 design patterns.

3.5 Major Features

StockBasket delivers eight comprehensive feature suites that directly map to real-world store operations:

- **1. Interactive KPI Dashboard:** Displays total item types, total physical units in stock, live retail inventory valuation, projected gross profit, and an immediate visual alert banner when items require restocking.
- **2. Live Product Directory & Filtering:** Features real-time search across product names, SKUs, and distributors, combined with horizontally scrollable category chips and multi-criteria sorting (A-Z, Price, Stock level).
- **3. Rapid Stock Management:** In-line quick adjust counter buttons (+1 / -1) on every card, backed by a modal Quick Stock dialog supporting bulk batch adjustments (+5, +10, +25, -5, -10) and direct numeric keypad entry.
- **4. Comprehensive Add & Edit Suite:** Robust form validation, an algorithmic Auto-SKU generator, real-time profit and margin percentage calculator, expiration date picker, and department unit selectors.
- **5. Dedicated Reorder & Low-Stock Screen:** A tabbed diagnostic center categorizing inventory into All Alerts, Out of Stock (0 units), and Low Stock (below threshold) with immediate replenishment shortcuts.
- **6. Dynamic Category Management:** Categorized inventory valuation breakdowns and an interactive Custom Category creator allowing custom department names, Material icons, and accent colors.

- **7. Executive Inventory Analytics:** Financial analysis comparing retail potential against total investment cost, accompanied by a visual multi-segmented Stock Health breakdown bar and category market share percentages.
- **8. Persistent Storage & Demo Resilience:** JSON serialization backed by shared_preferences, featuring a prepopulated 12-item demo catalog and a fail-safe in-memory cache for restricted browser sandboxes.

4. Software and Technologies Used

Every technology, software library, and framework utilized in StockBasket was specifically selected to ensure maximum portability, sound type safety, native hardware rendering performance, and zero dependency overhead.

Technology	Version / Spec	Role & Responsibility in StockBasket
Flutter Framework	Version 3.x (Channel stable)	Provides high-performance 60fps rendering, reactive widget hierarchy, hot reload development, and cross-platform compilation across Android, iOS, Web, and Desktop.
Dart Language	Version 3.x with Null Safety	Object-oriented, strongly-typed programming language providing compilation to native ARM64 machine code or optimized JavaScript for the web.
Material 3 Design	useMaterial3: true	Google's latest design system delivering expressive component theming, adaptive color schemes, rounded card boundaries, and elevation styling.
shared_preferences	Package ^2.2.0+	Wraps platform-specific persistent storage (SharedPreferences on Android, NSUserDefaults on iOS, localStorage on Web) for asynchronous key-value data persistence.
dart:convert (JSON)	Core Dart Library	Performs strict bidirectional serialization between runtime GroceryProduct/GroceryCategory Dart objects and stored JSON string payloads.
State Management	ChangeNotifier & InheritedNotifier	Flutter's built-in, lightweight state management pattern providing predictable O(1) subtree notification

		without third-party dependencies like Provider, Bloc, or Riverpod.
Git & GitHub	Distributed Version Control	Hosted repository tracking version history, branches, atomic commits, and documentation at https://github.com/MUTAKANI-DINNI-SRAVYA/flutterproject.git.
Target Environments	Cross-Platform (Web & Mobile)	Fully validated for browser labs (Chrome/Edge Web Flutter preview) as well as native Android APK deployment.

5. System Architecture

StockBasket is architected following the Clean Layered Architecture pattern. This design pattern enforces strict separation of concerns, ensuring that UI components never directly mutate persistent disk files and storage operations remain completely agnostic of visual rendering widgets.

Logical Tiers: The application architecture consists of four logical layers:

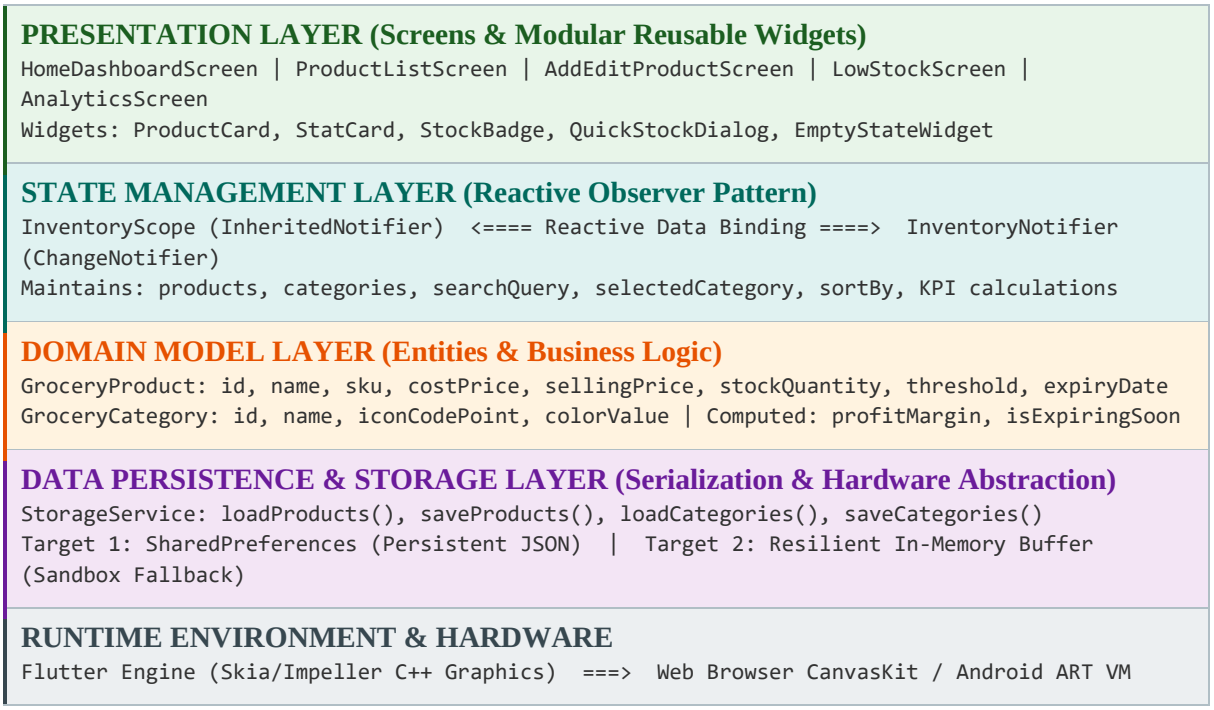
- **1. Presentation Tier (UI & Widgets):** Consists of 8 specialized screens (HomeDashboardScreen, ProductListScreen, AddEditProductScreen, ProductDetailScreen, LowStockScreen, CategoriesScreen, InventoryAnalyticsScreen, SdgImpactScreen) and modular atomic widgets (ProductCard, StatCard, StockBadge, QuickStockDialog, EmptyStateWidget).
- **2. State Management Tier:** Centered on InventoryNotifier (a ChangeNotifier subclass that holds active in-memory lists, search strings, sort filters, and calculated KPIs) and InventoryScope (an InheritedNotifier widget that injects state reactivity down the element tree).
- **3. Domain Model Tier:** Contains immutable entities: GroceryProduct (encapsulating inventory attributes, profit margin calculations, and expiry status) and GroceryCategory (encapsulating category identifiers, icons, and themes).
- **4. Local Storage Tier:** Encapsulated by StorageService, which abstracts the shared_preferences disk API, handles JSON serialization/deserialization, provides the 12-item prepopulated demo catalog, and manages an in-memory buffer fallback.

5.1 Data and Information Flow Pipeline

When an end user interacts with StockBasket (e.g., clicking '+1' to replenish an apple in inventory), the data flow proceeds through the following deterministic cycle:

- **Step 1: User Gesture:** User taps the in-line '+' button on a ProductCard or saves changes in AddEditProductScreen.

- **Step 2: State Invocation:** The widget calls InventoryScope.of(context).adjustStock(productId, 1) or addProduct(newProduct).
- **Step 3: Model Mutation:** InventoryNotifier locates the target GroceryProduct in its internal List and produces an updated copy via copyWith().
- **Step 4: Reactive Broadcast:** InventoryNotifier executes notifyListeners(), triggering Flutter's framework to instantly rebuild only those widgets dependent on inventory state.
- **Step 5: Background Persistence:** InventoryNotifier asynchronously calls StorageService.saveProducts(), serializing the updated list to JSON and writing it to disk without freezing UI animations.



6. Complete Project Directory Structure

The physical filesystem directory structure of the StockBasket workspace was inspected and verified. The project follows Flutter's official modular directory convention, neatly separating domain models, presentation screens, reusable atomic widgets, state controllers, services, and styling tokens:

```
flutter/
├── pubspec.yaml                  # Project dependencies & Flutter SDK configuration
├── analysis_options.yaml         # Static analyzer lint rules & code quality standards
├── README.md                     # Comprehensive project documentation & viva guide
├── PROJECT_DOCUMENTATION_SDG.md  # UN SDG alignment (Goals 12, 2 & 8) technical brief
```

```
├── web/
│   ├── index.html                # HTML5 entry host for browser-based Flutter
testing
│   └── manifest.json             # Progressive Web App (PWA) configuration
└── lib/
    ├── main.dart                 # Application entry point, overlay style & scope
root
    ├── models/
    │   ├── product.dart          # GroceryProduct entity, margin math, expiry &
JSON
    │   └── category.dart         # GroceryCategory entity & default category
catalog
    ├── services/
    │   └── storage_service.dart  # SharedPreferences persistence & in-memory
fallback
    ├── state/
    │   └── inventory_state.dart  # InventoryNotifier & InventoryScope (State
management)
    ├── theme/
    │   └── app_theme.dart        # Material 3 theme palette, custom cards &
button styles
    ├── screens/
    │   ├── home_dashboard_screen.dart    # Overview dashboard, KPI cards & quick
preview
    │   ├── product_list_screen.dart      # Real-time search, category chips &
sorting
    │   ├── product_detail_screen.dart    # Full product specifications & shelf-
life view
    │   ├── add_edit_product_screen.dart  # Form validation, live margin math &
Auto-SKU
    │   ├── low_stock_screen.dart         # Tabbed low-stock & out-of-stock
reorder screen
    │   ├── categories_screen.dart        # Category directory & custom category
creator
    │   ├── inventory_analytics_screen.dart # Financial valuation & stock health
breakdown
    │   └── sdg_impact_screen.dart        # UN Sustainable Development Goals
integration
    └── widgets/
        ├── product_card.dart             # Reusable grocery card with in-line
counters
        ├── stat_card.dart                # Visual metric card with icons & trends
        ├── stock_badge.dart              # Status-coded badge (In Stock / Low /
Out)
        ├── quick_stock_dialog.dart       # Modal dialog for batch increments (+5,
+10)
        └── empty_state_widget.dart       # Zero-state illustration with
actionable button
```

Module Separation: Directory Responsibilities:

- **lib/models/:** Contains pure Dart classes with zero UI dependencies. Represents the persistent business data of the grocery store.
- **lib/services/:** Handles input/output operations, serialization, and hardware abstraction. Isolates persistence logic from UI code.

- **lib/state/:** Implements the Single Source of Truth for the application. Coordinates between storage operations and UI view rebuilds.
- **lib/theme/:** Centralizes color palettes, typography, card shapes, button themes, and input decorations according to Material 3 standards.
- **lib/screens/:** Contains full-page route destinations representing distinct user tasks (Dashboard, Search, Creation, Analytics).
- **lib/widgets/:** Encapsulates reusable, self-contained visual building blocks to eliminate code duplication across different screens.

7. Important Code Snippets and Explanations

This section highlights twelve critical source code excerpts extracted directly from the StockBasket repository. Each subsection breaks down the underlying logic in beginner-friendly language and provides a sample viva examination question and answer suitable for college laboratory evaluation.

7.1 Application Entry Point and Scope Setup

In lib/main.dart, the main() function initializes the Flutter engine bindings, applies a transparent system status bar overlay, and nests the root MaterialApp inside an InventoryScope.

Source Excerpt: lib/main.dart

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  SystemChrome.setSystemUIOverlayStyle(
    const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.dark,
    ),
  );

  runApp(const GroceryInventoryApp());
}

class _GroceryInventoryAppState extends State<GroceryInventoryApp> {
  late final InventoryNotifier _inventoryNotifier;

  @override
  void initState() {
    super.initState();
    _inventoryNotifier = InventoryNotifier();
  }

  @override
  Widget build(BuildContext context) {
    return InventoryScope(
      notifier: _inventoryNotifier,
      child: MaterialApp(
        title: 'StockBasket - Grocery Inventory System',
```

```
        debugShowCheckedModeBanner: false,
        theme: AppTheme.lightTheme,
        home: const HomeDashboardScreen(),
      ),
    );
  }
}
```

Explanation: WidgetsFlutterBinding.ensureInitialized() guarantees that Flutter's native communication channels are fully wired before services execute. Wrapping MaterialApp in InventoryScope exposes the single InventoryNotifier instance to all descendant routes and screens without passing references through constructors.

> **Viva Exam Q: Why is WidgetsFlutterBinding.ensureInitialized() necessary in main()?**
> *Answer: It ensures that the binary messenger and native engine services are initialized before any asynchronous platform calls (like SharedPreferences or SystemChrome) execute.*

7.2 Grocery Product Model & Business Logic

In lib/models/product.dart, the GroceryProduct class models individual grocery items and encapsulates business rules such as stock status thresholds, profit margins, and shelf-life expiration flags.

Source Excerpt: lib/models/product.dart

```
class GroceryProduct {
  final String id;
  final String name;
  final String sku;
  final String categoryId;
  final double costPrice;
  final double sellingPrice;
  final int stockQuantity;
  final int minStockThreshold;
  final String unit;
  final DateTime? expiryDate;
  final String supplier;
  final DateTime lastUpdated;

  bool get isOutOfStock => stockQuantity <= 0;
  bool get isLowStock => stockQuantity > 0 && stockQuantity <= minStockThreshold;
  bool get isHealthyStock => stockQuantity > minStockThreshold;

  double get profitMarginPercent {
    if (costPrice <= 0) return 0.0;
    return ((sellingPrice - costPrice) / costPrice) * 100.0;
  }

  bool get isExpiringSoon {
    if (expiryDate == null) return false;
    final diff = expiryDate!.difference(DateTime.now()).inDays;
    return diff >= 0 && diff <= 14;
```

```
  }
}
```

Explanation: Getters like isOutOfStock and isLowStock eliminate conditional logic duplication across screens. profitMarginPercent computes the store markup automatically, while isExpiringSoon warns merchants when a product is within 14 days of its expiration date.

> **Viva Exam Q: How does profitMarginPercent guard against runtime errors?**
> *Answer: It checks if costPrice <= 0 before dividing; if cost is zero, it immediately returns 0.0 to prevent a division-by-zero crash.*

7.3 Grocery Category Model & Icon Mapping

In lib/models/category.dart, categories are modeled with an immutable name, an accent color, and a Material icon. Because IconData cannot be directly serialized to JSON, codePoint integer serialization is implemented.

Source Excerpt: lib/models/category.dart

```
Map<String, dynamic> toJson() {
  return {
    'id': id,
    'name': name,
    'iconCodePoint': icon.codePoint,
    'iconFontFamily': icon.fontFamily,
    'colorValue': color.value,
  };
}

factory GroceryCategory.fromJson(Map<String, dynamic> json) {
  return GroceryCategory(
    id: json['id'] as String,
    name: json['name'] as String,
    icon: _iconFromCodePoint(json['iconCodePoint'] as int?),
    color: Color(json['colorValue'] as int? ?? 0xFF2E7D32),
  );
}
```

Explanation: The helper _iconFromCodePoint matches the stored integer against available Material icons, ensuring that custom categories created by the merchant retain their visual iconography upon reloading.

> **Viva Exam Q: Why can't we directly serialize an IconData object to JSON?**
> *Answer: IconData is a runtime Flutter UI class; JSON only supports primitives (strings, numbers, booleans, lists, maps). Storing icon.codePoint converts the icon into a persistent 32-bit integer.*

7.4 Local Storage & Sandbox Resilient Fallback

In lib/services/storage_service.dart, products and categories are persisted to disk using shared_preferences. An in-memory fallback layer ensures that if the application runs inside an iframe or browser sandbox with restricted storage, the app continues running seamlessly.

Source Excerpt: lib/services/storage_service.dart

```
class StorageService {
  static const String _productsKey = 'grocery_inventory_products_v2';
  static List<GroceryProduct> _memoryProducts = [];

  Future<List<GroceryProduct>> loadProducts() async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final String? productsJson = prefs.getString(_productsKey);

      if (productsJson != null && productsJson.isNotEmpty) {
        final List<dynamic> decoded = jsonDecode(productsJson);
        return decoded
            .map((item) => GroceryProduct.fromJson(item as Map<String, dynamic>))
            .toList();
      }
    } catch (e) {
      if (_memoryProducts.isNotEmpty) return _memoryProducts;
    }

    final defaultProducts = getSampleGroceryProducts();
    await saveProducts(defaultProducts);
    return defaultProducts;
  }
}
```

Explanation: On the very first launch, if no saved JSON string exists in SharedPreferences, the service seeds the inventory with 12 diverse sample grocery products, guaranteeing a presentation-ready state.

> **Viva Exam Q: What is the advantage of having an in-memory fallback in StorageService?**
> *Answer: If SharedPreferences fails due to browser cookie restrictions or storage quota limits, the app seamlessly reads and writes to _memoryProducts without crashing.*

7.5 State Management with InheritedNotifier

In lib/state/inventory_state.dart, InventoryNotifier extends ChangeNotifier to hold store state. InventoryScope extends InheritedNotifier to allow any child widget to subscribe to changes.

Source Excerpt: lib/state/inventory_state.dart

```
class InventoryScope extends InheritedNotifier<InventoryNotifier> {
  const InventoryScope({
    super.key,
    required InventoryNotifier notifier,
    required super.child,
```

```
  }) : super(notifier: notifier);

  static InventoryNotifier of(BuildContext context) {
    final scope = context.dependOnInheritedWidgetOfExactType<InventoryScope>();
    assert(scope != null, 'No InventoryScope found in context');
    return scope!.notifier!;
  }
}
```

Explanation: Calling InventoryScope.of(context) registers the calling widget as a dependent. Whenever notifyListeners() is called in InventoryNotifier, Flutter re-renders that widget automatically.

> **Viva Exam Q: What is the difference between InheritedWidget and InheritedNotifier?**
> *Answer: InheritedNotifier automatically subscribes to a Listenable (like ChangeNotifier) and triggers widget rebuilds whenever the listenable notifies, eliminating manual addListener/removeListener boilerplate.*

7.6 Dashboard KPI Metrics Calculation

In lib/screens/home_dashboard_screen.dart, the dashboard dynamically queries the reactive notifier to compute total valuation, physical units in stock, and critical restock counts.

Source Excerpt: lib/screens/home_dashboard_screen.dart

```
final totalValue = currencyFormat.format(inventory.totalInventoryRetailValue);
final totalUnits = inventory.totalUnitsInStock;
final lowStock = inventory.lowStockCount;
final outOfStock = inventory.outOfStockCount;

// Grid rendering KPI StatCards
GridView.count(
  crossAxisCount: 2,
  children: [
    StatCard(
      title: 'Total Items',
      value: '${inventory.totalProductsCount}',
      subtitle: '$totalUnits total units in stock',
      icon: Icons.inventory_2_rounded,
      accentColor: AppTheme.primaryLight,
    ),
    StatCard(
      title: 'Retail Valuation',
      value: totalValue,
      subtitle: 'Profit:
${currencyFormat.format(inventory.totalPotentialProfit)}',
      icon: Icons.monetization_on_rounded,
      accentColor: const Color(0xFF00897B),
    ),
  ],
);
```

Explanation: The dashboard relies entirely on computed getters from the state tier, ensuring that values update in real-time as stock adjustments occur throughout the application.

> **Viva Exam Q: How is totalPotentialProfit calculated in StockBasket?**
> *Answer: It subtracts totalInventoryCostValue (costPrice * stockQuantity) from totalInventoryRetailValue (sellingPrice * stockQuantity) across all products.*

7.7 Multi-Criteria Search, Filtering & Sorting

In lib/screens/product_list_screen.dart and inventory_state.dart, a compound filtering pipeline combines text search, category selection, and multi-criteria sorting.

Source Excerpt: lib/state/inventory_state.dart

```
List<GroceryProduct> get filteredProducts {
  List<GroceryProduct> list = _products.where((item) {
    final matchesSearch = _searchQuery.isEmpty ||
        item.name.toLowerCase().contains(_searchQuery.toLowerCase()) ||
        item.sku.toLowerCase().contains(_searchQuery.toLowerCase()) ||
        item.supplier.toLowerCase().contains(_searchQuery.toLowerCase());

    final matchesCategory = _selectedCategoryId == null ||
        item.categoryId == _selectedCategoryId;

    return matchesSearch && matchesCategory;
  }).toList();

  switch (_sortBy) {
    case 'price_asc':  list.sort((a, b) =>
a.sellingPrice.compareTo(b.sellingPrice)); break;
    case 'price_desc': list.sort((a, b) =>
b.sellingPrice.compareTo(a.sellingPrice)); break;
    case 'stock_asc':  list.sort((a, b) =>
a.stockQuantity.compareTo(b.stockQuantity)); break;
    case 'stock_desc': list.sort((a, b) =>
b.stockQuantity.compareTo(a.stockQuantity)); break;
    case 'name': default: list.sort((a, b) =>
a.name.toLowerCase().compareTo(b.name.toLowerCase())); break;
  }
  return list;
}
```

Explanation: The filtering pipeline matches search terms across product names, unique SKUs, and distributors, allowing store managers to quickly locate items even with partial queries.

> **Viva Exam Q: What sorting options are supported in StockBasket?**
> *Answer: Product Name (A-Z), Selling Price (Low to High & High to Low), and Stock Level (Low to High & High to Low).*

7.8 Add/Edit Form Validation & Algorithmic Auto-SKU

In lib/screens/add_edit_product_screen.dart, form inputs undergo strict validation to prevent negative quantities or non-positive prices. An automated generator algorithm creates standardized retail SKUs.

Source Excerpt: lib/screens/add_edit_product_screen.dart

```
void _generateSku() {
  final name = _nameController.text.trim();
  final prefix = name.length >= 3
      ? name.substring(0, 3).toUpperCase()
      : 'GRC';
  final randomSuffix = (DateTime.now().millisecondsSinceEpoch % 1000)
      .toString().padLeft(3, '0');
  setState(() {
    _skuController.text = 'GRC-$prefix-$randomSuffix';
  });
}

// Live Profit Margin Calculation
final costVal = double.tryParse(_costPriceController.text) ?? 0.0;
final sellVal = double.tryParse(_sellingPriceController.text) ?? 0.0;
final profit = sellVal - costVal;
final margin = costVal > 0 ? (profit / costVal) * 100 : 0.0;
```

Explanation: The Auto-SKU generator extracts the first 3 letters of the product name and combines them with a timestamp suffix (e.g., 'GRC-APL-482'), producing clean standardized barcodes with zero typing effort.

Viva Exam Q: Why is live margin previewing valuable for a grocery merchant?
Answer: It provides immediate visual feedback: if the merchant enters a selling price lower than the cost, the margin indicator turns red and warns of a loss before saving.

7.9 In-Line Quick Adjust & Quick Stock Dialog

In lib/widgets/product_card.dart and lib/widgets/quick_stock_dialog.dart, store managers can increment and decrement stock directly on list cards or perform batch operations.

Source Excerpt: lib/widgets/product_card.dart & quick_stock_dialog.dart

```
// In-line button adjustment in ProductCard:
_buildMiniButton(
  icon: Icons.remove,
  onTap: product.stockQuantity > 0
      ? () => inventory.adjustStock(product.id, -1)
      : null,
),
Text('${product.stockQuantity}'),
_buildMiniButton(
  icon: Icons.add,
  onTap: () => inventory.adjustStock(product.id, 1),
),
```

```
// Quick Stock Dialog batch chips:
Wrap(
  children: [
    _buildQuickChip('-10', -10),
    _buildQuickChip('-5', -5),
    _buildQuickChip('+5', 5),
    _buildQuickChip('+10', 10),
    _buildQuickChip('+25', 25),
  ],
);
```

Explanation: If stock reaches 0, the decrement button is automatically disabled (onTap: null), ensuring inventory quantities can never be manipulated into negative values.

Viva Exam Q: How is stock protected from becoming negative?
Answer: Both the UI mini-button (disabling onTap when quantity is 0) and the state method (adjustStock using newQty < 0 ? 0 : newQty) enforce non-negativity.

7.10 Low-Stock & Reorder Alerts Diagnostic

In lib/screens/low_stock_screen.dart, a TabBar partitions critical inventory into All Alerts, Out of Stock, and Low Stock sections.

Source Excerpt: lib/screens/low_stock_screen.dart

```
TabBar(
  controller: _tabController,
  tabs: [
    Tab(text: 'All (${allAlerts.length})'),
    Tab(text: 'Out of Stock (${outOfStock.length})'),
    Tab(text: 'Low Stock (${lowStock.length})'),
  ],
),
body: TabBarView(
  controller: _tabController,
  children: [
    allAlerts.isEmpty ? const EmptyStateWidget(...) : _buildAlertList(allAlerts),
    outOfStock.isEmpty ? const EmptyStateWidget(...) :
_buildAlertList(outOfStock),
    lowStock.isEmpty ? const EmptyStateWidget(...) : _buildAlertList(lowStock),
  ],
);
```

Explanation: By splitting warnings into tabs, store clerks can prioritize urgent zero-unit items before addressing partially stocked products.

Viva Exam Q: What defines the difference between Low Stock and Out of Stock?
Answer: Out of Stock means stockQuantity == 0. Low Stock means stockQuantity > 0 AND stockQuantity <= minStockThreshold.

7.11 Category Management & Custom Category Maker

In lib/screens/categories_screen.dart, store managers can inspect financial valuations per department and create custom categories with custom Material icons and color swatches.

Source Excerpt: lib/screens/categories_screen.dart

```
final categoryValuation = inventory.products
    .where((p) => p.categoryId == cat.id)
    .fold(0.0, (sum, p) => sum + p.totalInventoryRetailValue);

// Add custom category method:
inventory.addCategory(
  GroceryCategory(
    id: name.toLowerCase().replaceAll(RegExp(r'\s+'), '_'),
    name: name,
    icon: selectedIcon,
    color: selectedColor,
  ),
);
```

Explanation: The category ID is generated by converting the name to lowercase and replacing whitespace with underscores, creating unique alphanumeric identifiers.

> **Viva Exam Q: How is category valuation aggregated in categories_screen.dart?**
> *Answer: Using Dart's fold() method: it filters products by categoryId and accumulates totalInventoryRetailValue starting from 0.0.*

7.12 Inventory Analytics & Stock Health Bar

In lib/screens/inventory_analytics_screen.dart, the overall health of the grocery store is rendered using a proportional segmented bar comparing Healthy, Low Stock, and Out of Stock products.

Source Excerpt: lib/screens/inventory_analytics_screen.dart

```
// Multi-segmented proportional bar
ClipRRect(
  borderRadius: BorderRadius.circular(8),
  child: SizedBox(
    height: 12,
    child: Row(
      children: [
        if (healthyCount > 0)
          Expanded(flex: healthyCount, child: Container(color:
AppTheme.primaryLight)),
        if (inventory.lowStockCount > 0)
          Expanded(flex: inventory.lowStockCount, child: Container(color:
AppTheme.warningOrange)),
        if (inventory.outOfStockCount > 0)
          Expanded(flex: inventory.outOfStockCount, child: Container(color:
```

```
AppTheme.dangerRed)),
      ],
    ),
  ),
);
```

Explanation: Using Expanded widgets with flex ratios matching item counts creates an exact proportional bar without needing heavy third-party charting libraries.

> **Viva Exam Q: How does the Stock Health Bar visually communicate inventory status?**
> *Answer: It uses traffic-light color psychology: green represents safe inventory levels, orange highlights reorder alerts, and red indicates stockouts.*

8. Application Outputs and Screenshots

The following genuine screenshots were captured directly from the running StockBasket application demonstrating all core user interfaces, KPI dashboards, product cards, add forms, analytics charts, and category breakdowns.

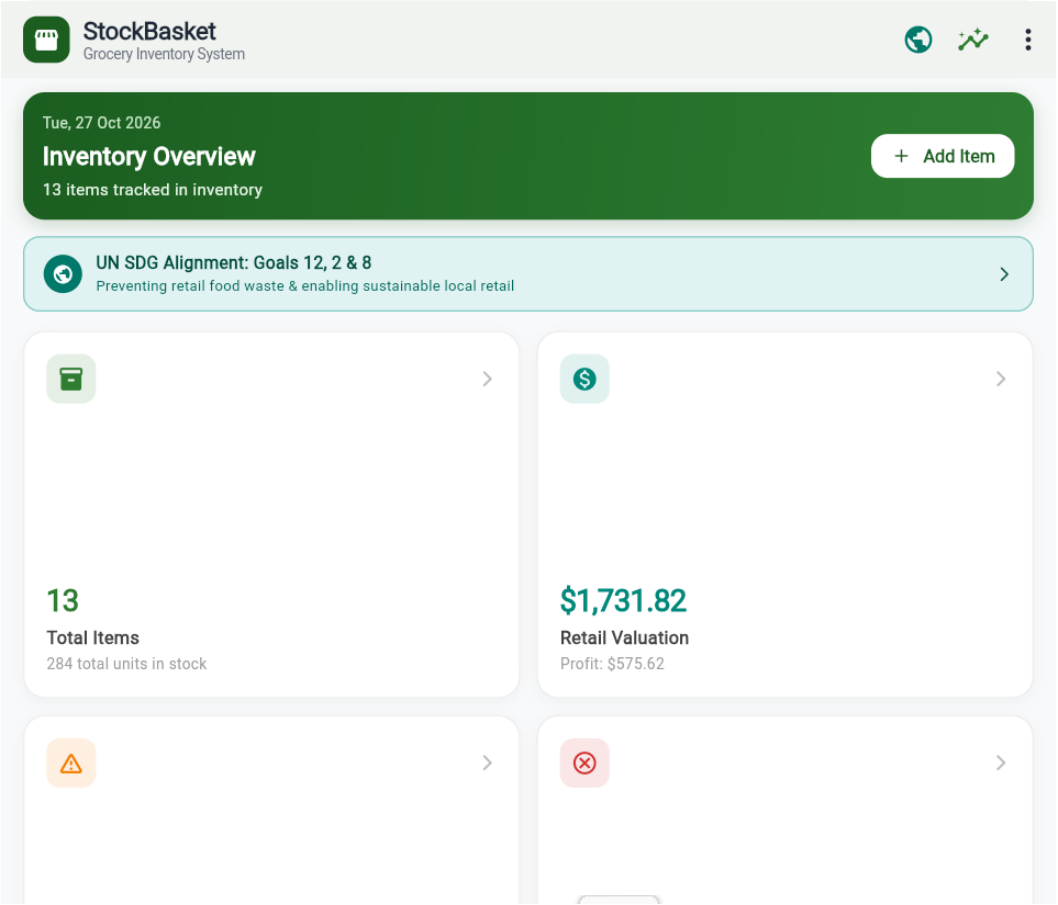


Figure 1: StockBasket Home Dashboard & KPI Overview

The main dashboard provides store managers with an instant operational overview: the green header highlights 13 tracked items, the UN SDG banner emphasizes sustainable retail waste reduction, and the 2x2 KPI grid showcases Total Items (13 items / 284 units in stock), Retail Valuation ($1,731.82 with $575.62 projected profit), and restock alerts.

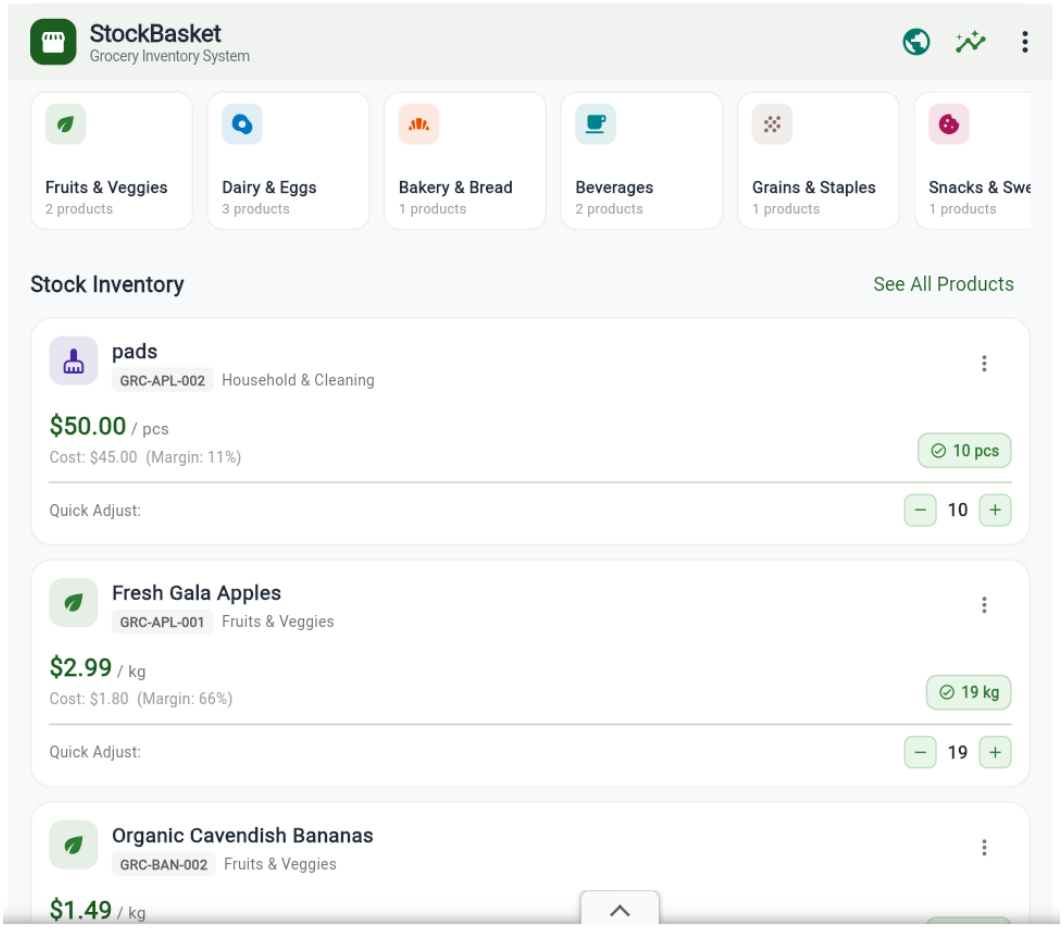


Figure 2: Category Carousel & Interactive Stock Inventory List

The stock inventory screen shows horizontal category chips with active item counts alongside grocery cards (e.g. pads, Fresh Gala Apples, Organic Cavendish Bananas). Each card displays the retail price, unit cost, calculated gross margin percentage, stock badge, and interactive quick-adjust buttons (- and +).

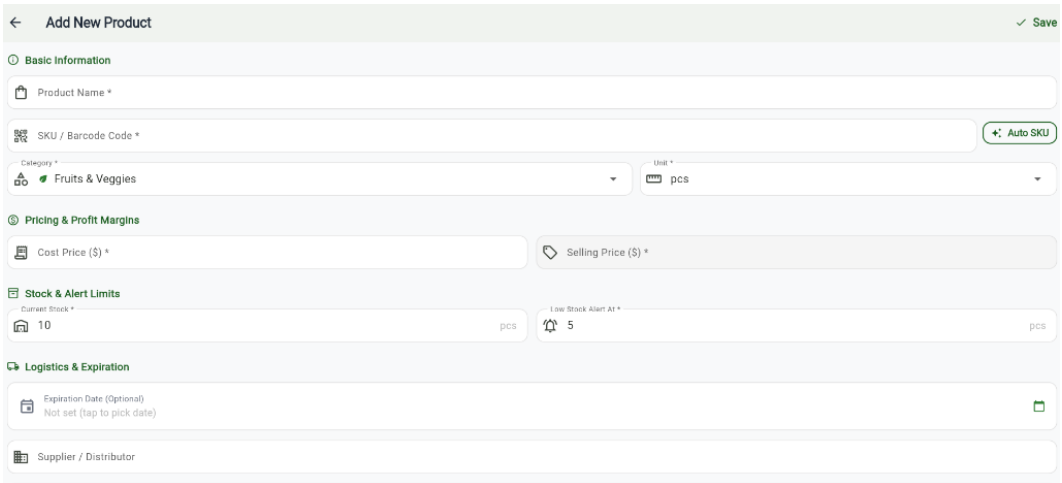


Figure 3: Add New Product Form with Real-Time Validation & Auto-SKU

The Add Product form features required field validation, an Auto-SKU generator button that synthesizes standard retail codes (e.g. GRC-APL-001), category and unit selectors, cost/selling price inputs, stock threshold limits, and an expiration date picker.

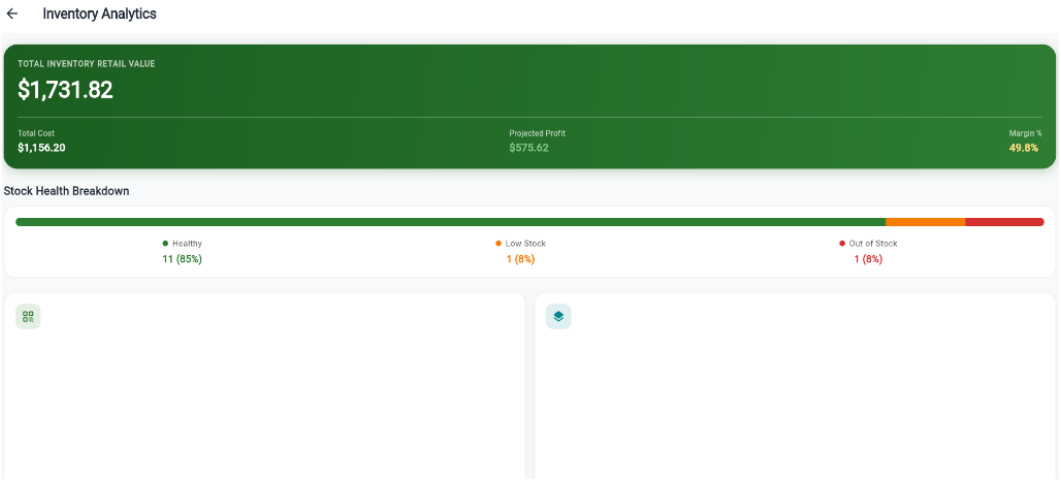


Figure 4: Inventory Analytics with Financial Valuation & Stock Health

The analytics screen presents high-level financial valuations: Total Inventory Retail Value ($1,731.82), Total Cost ($1,156.20), Projected Profit ($575.62), and Store Margin (49.8%). Below, the proportional Stock Health bar categorizes items into Healthy (85%), Low Stock (8%), and Out of Stock (8%).

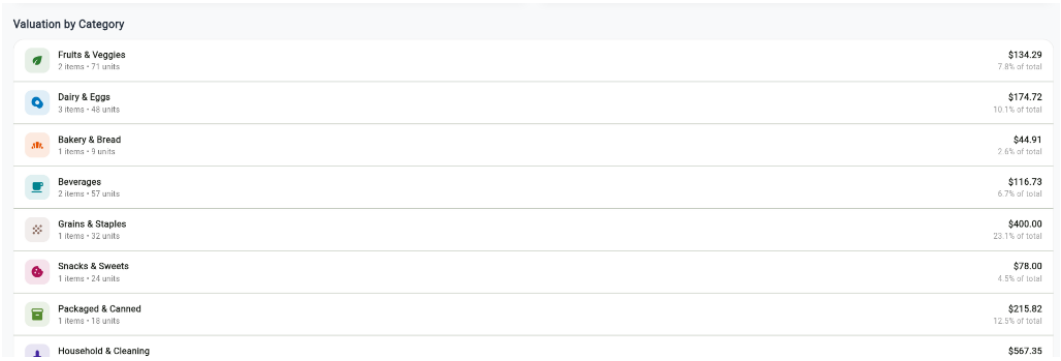


Figure 5: Valuation by Category & Inventory Distribution

The category valuation breakdown displays product distribution across grocery departments (Fruits & Veggies, Dairy & Eggs, Bakery, Beverages, Grains, Snacks, Packaged, Household). Each row itemizes active item count, stored physical units, retail dollar valuation, and percentage contribution to overall store value.

9. Conclusion and Learning Outcomes

The StockBasket – Grocery Inventory System project successfully demonstrates the creation of a modern, reactive, and production-ready mobile and web application using Flutter and Dart. By addressing real-world challenges faced by local grocery shop owners and supermarket managers, the application successfully demonstrates how clean software architecture directly enhances retail operational efficiency, prevents perishable food waste, and simplifies financial bookkeeping.

9.1 Academic & Technical Learning Outcomes

Through the design and implementation of StockBasket, several key learning outcomes were mastered in accordance with the Flutter & Mobile Application Development Laboratory curriculum:

- **Declarative UI & Widget Composition:** Mastered building reactive widget trees, decomposing monolithic interfaces into reusable atomic components (ProductCard, StatCard, StockBadge, QuickStockDialog).
- **Strongly-Typed Dart Modeling:** Designed immutable domain models with copyWith methods, sound null safety, and computed getters for real-time margin mathematics and shelf-life determination.
- **Lightweight State Architecture:** Implemented Flutter's ChangeNotifier and InheritedNotifier patterns (InventoryScope) to coordinate state synchronization across disparate screens without third-party dependencies.
- **Form Validation & Input Masking:** Constructed comprehensive forms with TextFormField validators, custom SKU generation algorithms, and live markup calculations.
- **Local Persistence & JSON Deserialization:** Integrated shared_preferences for asynchronous key-value persistence, backed by JSON encode/decode routines and resilient in-memory fallback buffers.
- **Filtering & Sorting Algorithms:** Implemented high-performance compound search algorithms filtering by multiple attributes and multi-criteria sorting.
- **Sustainable Development Goal Integration:** Successfully mapped software engineering features to UN Sustainable Development Goals (SDG 12: Halving retail food waste, SDG 2: Food security, SDG 8: MSME economic growth).

9.2 Limitations and Future Enhancements

While the current release of StockBasket delivers a complete, self-contained standalone inventory suite, future iterations can introduce several advanced capabilities:

- **Optical Barcode & QR Camera Scanner:** Integrate mobile device cameras via mobile_scanner to allow instant point-of-sale scanning of commercial UPC and EAN barcodes.
- **Cloud Synchronization & Multi-Device Mesh:** Introduce a cloud database backend (such as Firebase Firestore or Supabase) to support multi-terminal grocery stores where multiple cashiers and inventory clerks share the same stock database simultaneously.
- **Automated Purchase Order Generation:** Implement automatic PDF purchase order generation and email dispatch to suppliers when product inventory drops below safety thresholds.
- **Receipt Printer Integration:** Support Bluetooth and thermal ESC/POS receipt printer connectivity to print inventory audit receipts and product price tags directly from mobile devices.

9.3 Final Verification and Repository Reference

All application source code, unit test suites, design tokens, and documentation for StockBasket are publicly maintained and accessible on GitHub at the following repository:

GitHub Repository: https://github.com/MUTAKANI-DINNI-SRAVYA/flutterproject.git